

Rapport projet crypto - 2025-2026

MOULIN Antoine

3.1]

1) On teste l'application avec le chiffrement **rot13**, on a l'impression coté serveur que les messages sont chiffrés, or **il est très facile de deviner ces derniers** (Il suffit de réappliquer rot13 au message en question).

2) En réappliquant la fonction **rot13**, le serveur affiche **les messages en clair, ce qui prouve bien un manque de confidentialité.**

```
[Server -> Client 1]: READY
[Server -> Client 2]: READY
Server is now relaying messages between clients...
[Client 1 -> Server]: Uryyb
[MITM] Intercepted message from 1 to 2 : Hello
[Server -> Client 2]: Uryyb
```

3.2]

1) On a modifié "Client.java" pour prendre un "password" en argument:
exemple: *java Client test*

2) Dans Interceptor, on fait en sorte de générer une clé AES-128 en gardant les 16 premiers octets du SHA-256 du mot de passe choisi.

3) On implémente le chiffrement/déchiffrement des messages:

Coté serveur :

```
[Server -> Client 1]: READY
[Server -> Client 2]: READY
Server is now relaying messages between clients...
[Client 1 -> Server]: VZFqeQ/pEa5pKxC2T6hyoQ==:KqJxA6Vv212f8KzKT7QMUw==
[MITM] Intercepted message from 1 to 2 : IMSdrD/cRn5cXkP2G6ulbD==:XdWkN6Ii212s8XmXG7DZHj==
[Server -> Client 2]: VZFqeQ/pEa5pKxC2T6hyoQ==:KqJxA6Vv212f8KzKT7QMUw==
```

Coté clients :

```
hello
[Interceptor] Encrypting message: hello
You: hello
```

```
Chat session started!
Type your messages and press Enter to send.
Type 'exit' to quit.

[Interceptor] Decrypting message...
Other: hello
```

4) Grace au chiffrement **AES on** peut assurer la **confidentialité**, mais **pas l'intégrité** (on peut modifier le message intercepté) :

Serveur:

```
Server is now relaying messages between clients...
[Client 1 -> Server]: jPnRwdXzJGgtyvrThHPWuQ==:HzOLJTX5IHed6+JSOK3UYw==
[MITM] Intercepted: jPnRwdXzJGgtyvrThHPWuQ==:HzOLJTX5IHed6+JSOK3UYw==
[MITM] Modified message!
[Server -> Client 2]: jPnRwdXzJGAtyvrThHPWuQ==:HzOLJTX5IHed6+JSOK3UYw==
```

Clients:

```
Chat session started!
Type your messages and press Enter to send.
Type 'exit' to quit.

testt
[Interceptor] Encrypting message: testt
You: testt
```

```
Chat session started!
Type your messages and press Enter to send.
Type 'exit' to quit.

[Interceptor] Decrypting message...
Other: [Decryption failed: Given final block not properly padded. Such iss
ues can arise if a bad key is used during decryption.]
```

3.3]

1 et 2)

On remplace AES-CBC par GCM, toute modification du message chiffré est détectée au moment du déchiffrement. Le message est rejeté afin de garantir l'intégrité et la confidentialité :

```
Chat session started!
Type your messages and press Enter to send.
}Type 'exit' to quit.

[Interceptor] Decrypting message...
Other: [Decryption failed: message integrity check failed]
```

3.4]

1) On supprime l'utilisation des mots de passe et on remplace par une clé de session avec ECDH (un handshake).

```
Waiting for other client to connect...
Both clients connected!

--- Handshake Phase ---
[Interceptor] Starting handshake
[Interceptor] Handshake complete!
--- Handshake Complete ---

Chat session started!
Type your messages and press Enter to send.
Type 'exit' to quit.
```

2) On effectue une véritable attaque MITM en se faisant passer pour le client lors du handshake ECDH afin de déchiffrer en plus d'intercepter le message et de l'afficher en clair :

Coté Serveur :

```
[MITM] Replacing client 1 public key with MITM public key for client 2
[Server -> Client 2]: MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAE/g/acHn4uJicsipe80HdPA10ur1g/Qhs
oTRlvrDPMzy4nkEV7VubaB3Z5aS4efALsQd4UwBH6+h5BDzxleo4HQ==
[Client 1 -> Server]: F1PLfa3iOxjH3C3U:lgZq58mVaJIsC1ZBGwkoM06Q8ckgHA==
[MITM] Intercepted from 1 to 2: F1PLfa3iOxjH3C3U:lgZq58mVaJIsC1ZBGwkoM06Q8ckgHA==
[MITM] Message en clair de 1 vers 2 : Coucou
[Server -> Client 2]: LmrnXNKnoQdsasNW:1huNAPQnLMneFe0RbZ4CdwDS24Hlfw==
```

Coté Client :

```
Chat session started!
Type your messages and press Enter to send.
Type 'exit' to quit.

Coucou
[Interceptor] Encrypting message: Coucou
You: Coucou
```

3.5)

(POUR TESTER VEUILLEZ REGENERER LES CLES, ELLES SONT DANS UN GITIGNORE)

1) On génère une clé unique pour chaque client en utilisant openssl:

```
openssl ecparam -name prime256v1 -genkey -noout -out client1.key
openssl ec -in client1.key -pubout -out client1.pub
```

```
openssl ecparam -name prime256v1 -genkey -noout -out client2.key
openssl ec -in client2.key -pubout -out client2.pub
```

```
-rw----- 1 antoine antoine 227 Apr  9 15:53 client1.key
-rw-rw-r-- 1 antoine antoine 178 Apr  9 15:53 client1.pub
-rw----- 1 antoine antoine 227 Apr  9 15:53 client2.key
-rw-rw-r-- 1 antoine antoine 178 Apr  9 15:53 client2.pub
```

2) On implémente ce système lors du handshake ECDH, chaque client **signe sa clé publique avec sa clé privée**, la signature et les clés publiques sont ensuite envoyées à l'autre client afin qu'il vérifie la signature avant de calculer le secret partagé.

3) Meme avec l'ajout des **signatures**, l'**authenticité** n'est pas encore vérifiée et le **MITM** est toujours possible. Le serveur remplace les clés publiques et la signature par les siennes, établissant une clé de session avec chaque client pour déchiffrer les messages et les afficher en clair pour ensuite les rechiffrer et les retransmettre.

Coté Serveur:

```
[Client 1 -> Server]: 4UgY4L+IHeu7aRCi:C05DVNN6EpjoMwIQmvo02dqQXDvY4A==  
[MITM] Intercepted from 1 to 2: 4UgY4L+IHeu7aRCi:C05DVNN6EpjoMwIQmvo02dqQXDvY4A==  
[MITM] Message en clair de 1 vers 2 : Coucou  
[Server -> Client 2]: 03hHk56xNJXojTg3:Upeufrr++sxo0zEZRTGuUpwTxYLf/w==
```

Coté Clients:

```
Chat session started!  
Type your messages and press Enter to send.  
Type 'exit' to quit.  
  
Coucou  
[Interceptor] Encrypting message: Coucou  
You: Coucou
```

3.6]

1)

On génère la clé privée du CA (Certification d'autorité) avec openssl :

```
openssl ecparam -name prime256v1 -genkey -noout -out ca_raw.key  
openssl pkcs8 -topk8 -nocrypt -in ca_raw.key -out ca.key
```

On l'autosigne puisqu'il s'agit du certificat racine :

```
openssl req -new -x509 -key ca.key -out ca.crt -days 3650 \  
-subj "/CN=CryptoChat-CA"
```

2) On signe chaque les certificats des 2 clients grace à la CA:

```
antoine@antoine-Inspiron-7506-2n1:~/Desktop/Cyber/Crypto/CryptoChat$ openssl x509  
-req -in client1.csr -CA ca.crt -CAkey ca.key \  
-CAcreateserial -out client1.crt -days 3650  
Certificate request self-signature ok  
subject=CN = Client1
```

```
antoine@antoine-Inspiron-7506-2n1:~/Desktop/Cyber/Crypto/CryptoChat$ openssl verify -CAfile ca.crt client1.crt  
client1.crt: OK  
antoine@antoine-Inspiron-7506-2n1:~/Desktop/Cyber/Crypto/CryptoChat$ openssl verify -CAfile ca.crt client2.crt  
client2.crt: OK
```

3) Au démarrage d'un client, on lui passe maintenant en paramètre sa clé privée, son certificat et le certificat racine de l'AC afin de régler le problème d'authenticité :

```
java Client client1_pkcs8.key client1.crt ca.crt
java Client client2_pkcs8.key client2.crt ca.crt
```

4) Lors du handshake, chaque client **envoie maintenant sa clé publique signée ainsi que son certificat**. L'autre vérifie que le **certificat bien signé par la CA**, puis utilise la clé publique contenue dans le certificat **pour vérifier la signature** de la clé ECDH et on **peut afficher l'identité du client** :

```
--- Handshake Phase ---
[Interceptor] Starting handshake
[Interceptor] Remote identity: CN=Client1
[Interceptor] Handshake complete!
--- Handshake Complete ---
```

5) Puisque l'on peut vérifier l'authenticité, une attaque MITM peut dorénavant être détectée :

```
Waiting for other client to connect...
Both clients connected!

--- Handshake Phase ---
[Interceptor] Starting handshake
MITM DETECTE : certificat invalide ou falsifié
Disconnected from server.
antoine@antoine-Inspiron-7506-2n1:~/Desktop/Cyber/Cr
```

3.7]

1) Un attaquant **peut rejouer un ancien message chiffré** sans modification et ce dernier sera **valide et déchiffré correctement par le client**. On peut aussi les **supprimer sans aucune détection mise en place**. Notre protocole ne garantit ni l'unicité ni la continuité des messages.

2) Pour éviter cela, on pourrait par exemple mettre **un système de compteur de messages** (s'il y a **deux fois le même numéro** ou **un trou dans le compte**, on peut en déduire une attaque).

```
Chat session started!
Type your messages and press Enter to send.
Type 'exit' to quit.

[Interceptor] Decrypting message...
Other: salut
[Interceptor] Decrypting message...
Other: [ALERTE : message rejoue ou manquant]
```

3) Même si un attaquant **enregistre tous les messages chiffrés** puis récupère plus tard les **clés privées des clients**, il ne **pourra pas déchiffrer les anciennes communications** vu que chaque session utilise une **clé différente** générée avec une **clé ECDH éphémère**, donc les anciens messages **ne peuvent pas être déchiffrés**. On respecte donc la **Forward Secrecy**.

4]

Améliorations possibles

- ajouter un **compteur plus robuste** (timestamp)
- gérer la **perte de messages**
- améliorer la gestion des erreurs (messages plus clairs)

Ressources utilisées

- documentation **OpenSSL**
- cours de **cryptographie**
- assistance **via une IA** (ChatGPT) pour compréhension et code Java.